TuneBase

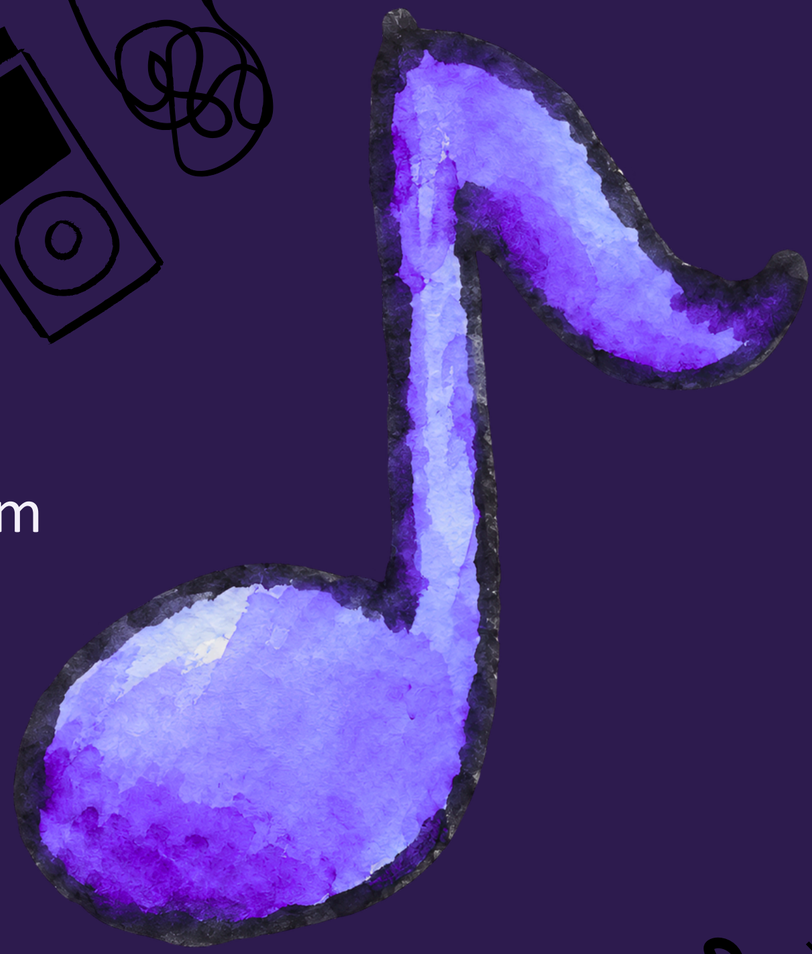
A Music Streaming Database Management System



DBMS Project | Flask + MySQL

Presented by : Navya Gupta
Sakshi



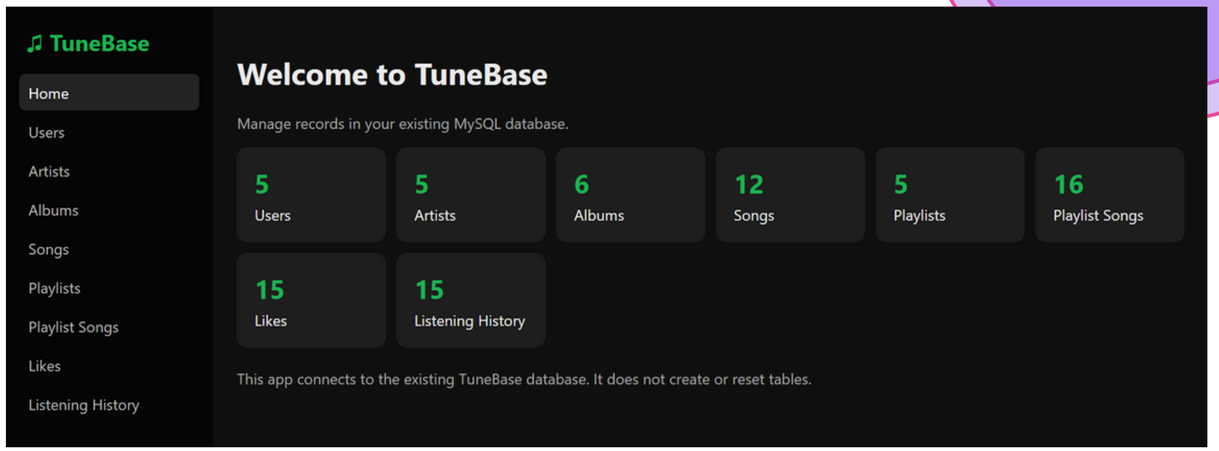




Project Overview

TuneBase is a database management application designed to organize music-related information. It stores users, artists, albums, songs, playlists, likes, and listening history

- Store songs, artists, albums, users and playlists in one relational database
- Record each user's likes and listening history
- Apply ER modelling, 3NF and integrity constraints
- Provide a Flask web interface for full CRUD



Technology Stack

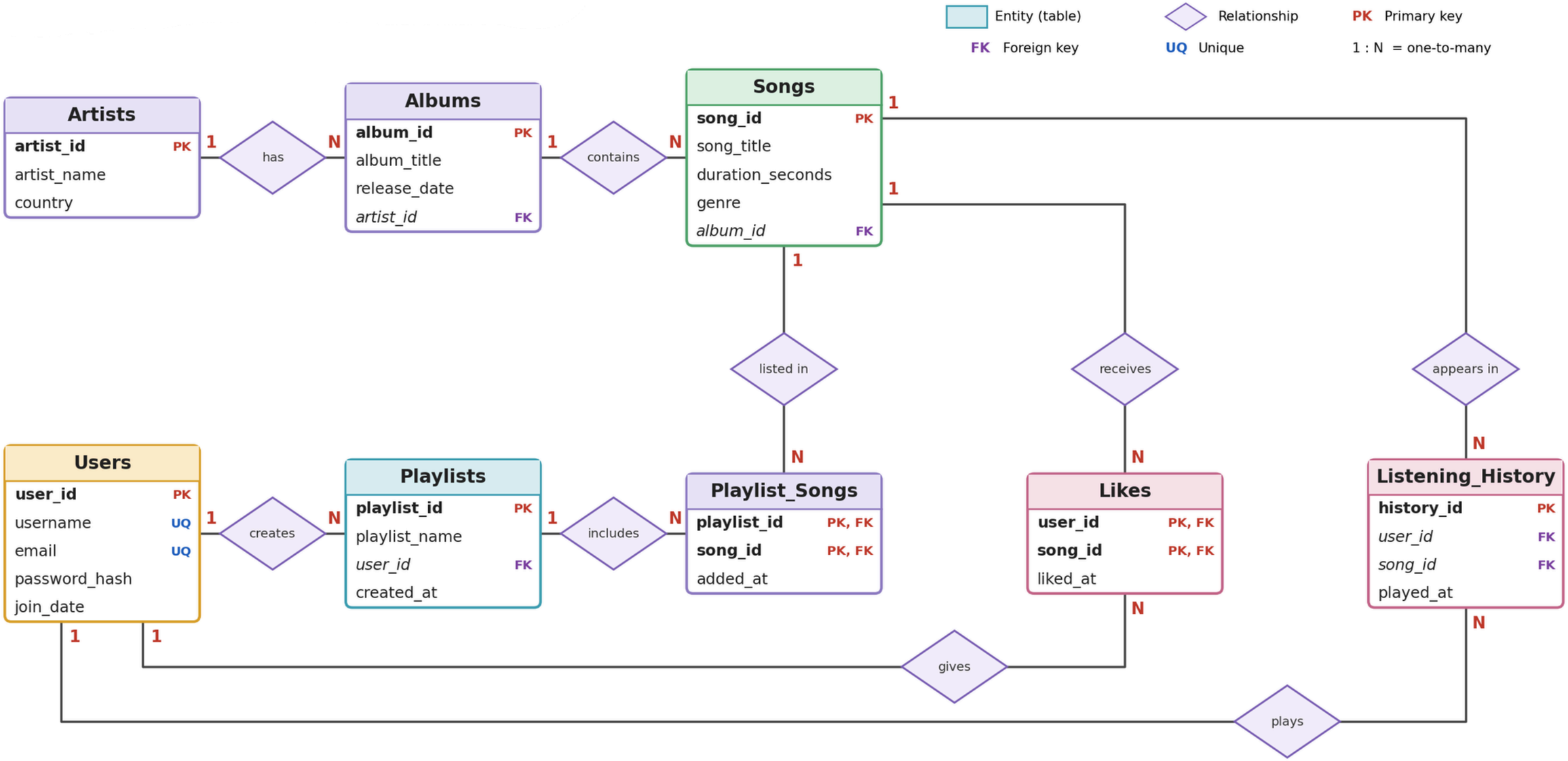
- Database: MySQL
- Local server environment: XAMPP
- Backend: Python with Flask
- Database connectivity: mysql-connector-python
- Frontend: HTML and CSS
- Database interface: phpMyAdmin

8
Tables

3NF
Normalized



Entity Relationship Diagram



Relationships & Cardinalities

Relationship	Cardinality	Implemented by
Artists - Albums	1 : many	artist_id in Albums
Albums - Songs	1 : many	album_id in Songs
Users - Playlists	1 : many	user_id in Playlists
Playlists - Songs	many : many	Junction table Playlist_Songs
Users - Songs (likes)	many : many	Junction table Likes
Users - Songs (plays)	1 : many each	Event table Listening_History



Schema & Data Integrity

1 Keys & uniqueness

Primary key on every table; UNIQUE username and email; composite keys stop duplicate likes and playlist rows.

2 Referential integrity

Foreign keys link all eight tables; ON DELETE CASCADE cleans up user-owned playlists, likes and history.



3 Validation

NOT NULL on required fields and CHECK (duration_seconds > 0) reject bad song data.

4 Automatic defaults

join_date, created_at, added_at, liked_at and played_at fill themselves in.

Normalization up to 3NF

1NF

Atomic values

Each table uses individual attributes rather than repeating groups. Each field is intended to hold a single value, and records are identifiable through keys.

2NF

Whole-key dependency

The schema uses primary keys to identify records. In the junction tables, such as Playlist_Songs and Likes, composite keys identify each relationship. Non-key attributes must depend on the entire key rather than only part of a composite key.

3NF

No transitive dependency

Artist, album and user details live once in their own tables; other tables keep only foreign keys.



The final schema should be checked for partial and transitive dependencies before claiming full 3NF compliance.

SQL Queries

Example:

4 plays for Bite Me, the most-played song

3 songs never played

1 artist with two albums: Coldplay

queries in queries.sql, from simple SELECTs to transactions

Basic retrieval
SELECT, WHERE, ORDER BY, LIKE

Joins
3-table joins: songs, playlists, likes

Aggregation
GROUP BY, HAVING, AVG, LEFT JOIN

Subqueries
Above-average length, never played

Views & transactions
SongDetails view, UPDATE, ROLLBACK

Flask Web Application

C **Create**
Add record form

R **Read**
Browse and search

U **Update**
Edit any row

D **Delete**
Confirm and remove

The Flask application provides a dashboard and table-specific pages for managing database records.

Features:

- Displaying records from database tables.
- Creating new records.
- Editing existing records.
- Deleting records.
- Searching table data.
- Selecting related records through foreign-key dropdowns.

Conclusion

What we delivered

TuneBase demonstrates the design and implementation of a relational database application for managing music-library information. It combines relational schema design, SQL queries, referential integrity, and a Flask-based CRUD interface. Final validation includes confirming CRUD functionality across all eight tables, verifying constraints, and submitting a complete ER diagram with cardinalities and participation constraints.

